Run full Terminal-Bench agentic RL on AWS

A hybrid solution: AgentCore microVMs, m5 EC2 instances and g6 GPU Instances, plus KernelBench

demo_hybrid_terminalbench
slime (Qwen3-4B, fully-async GRPO) on a g6e.48xlarge trainer

Data as of 4 October 2026 (us-west-2)

Agenda

01 Terminal-Bench
What it is, three tasks, one image per task, who runs where

02 KernelBench
GPU kernel tasks and their reward

03 Image sizes
Why one sandbox product is not enough

04 The hybrid solution
Components, pipeline, sessions and instances, m5 and g6

05 Pre-open
Hiding cold starts of 1 s to 2.5 min

06 Results and conclusion
Smoke runs, training run, what is solved

Terminal-Bench: 241 real terminal tasks, each graded by its own hidden tests

- **What it is:** a benchmark of tasks an agent solves **through a shell**: fix a broken system, configure a service, process data, attack a vulnerable app (laude-institute/terminal-bench).
- **Each task ships:** an instruction, a **Dockerfile** (its environment), a `docker-compose.yaml`, and **hidden pytest tests** plus a reference solution.
- **The reward** (RLVR, no LLM judge): after the agent stops, the tests are copied in and run; reward 1 if they all pass.
- **The agent:** Qwen3-4B with `bash`, `write_file` and `read_file`, up to 8 turns, running **inside** the task's sandbox and calling the trainer for each turn.

Table 1 **Corpus**

Tasks	n
single container (`client` only)	229
multi-container compose (client + sidecars)	12
total	**241**

In plain English: every task is a small broken computer plus a hidden exam; the agent fixes it with shell commands and the exam scores it.

Source: terminalbench/tasks/manifest.json (fetch_terminalbench.py, laude-institute/terminal-bench main), checked 4 Oct 2026.

Three tasks show why each needs its own environment and substrate

Table 2 **Example tasks**

Task	Instruction (abridged)	Environment → substrate
jq-data-processing easy, data processing	"Use jq to transform `/app/users.json`: keep active users, reformat the ISO-8601 last_login dates, write `/app/active_users_report.json`."	One container, small image (jq + data file). **AgentCore microVM**
postgres-csv-clean medium, file operations	"Connect to the Postgres DB from the compose config, import `customers.csv`, clean the data, export a cleaned CSV."	**Two containers**: `client` + a `postgres:16` sidecar on a private network. **m5 EC2** (compose project)
mteb-eval medium, data science	"Evaluate an embedding model from Hugging Face on STS12 with `mteb` so that cosine_spearman $>$ 0.7; store the model in the default cache."	One container, but a **6.7 GB** image (torch + models): over the 2 GB microVM cap. **m5 EC2**

In plain English: the environment decides the substrate: small and single-container goes to a microVM; several containers, x86-only or a big image goes to an m5.

Source: `terminalbench/tasks/manifest.json`, `images.csv`, `route_by_image.json`; checked 4 Oct 2026.

The route is data: 241 tasks → 236 routed → 193 with a trusted reward

Table 3 **Where each task goes and why**

Step	Tasks
Corpus (229 single + 12 compose)	241
microVM: arm64 build ships ($\leq$ 2 GB)	218
build failed / over 2 GB	7 / 4
m5: compose, x86-only, $>$ 2 GB, or microVM oracle failed	78
built on the m5 builder	71
Routed (**microVM** / **m5**)	165 / 71
Oracle passes (**microVM** / **m5**)	**163 / 30 = 193**

- **Every route comes from a measured result**: build status and size (`images.csv`), compose or not (`manifest.json`), and the reference solution run on the substrate (`oracle.csv`).
- **A task that fails its oracle on a microVM is retried on the m5**, which gives it root, raw networking and x86; it is kept where it passes.
- `image_routing.py` **refuses an unmapped task**: no silent fallback to the wrong sandbox.
- **193 usable vs 165** for the microVM-only demo (`demo_cpu_terminalbench`): 17 % more of the corpus.
- **Not rescued:** 7 m5 builds fail upstream (e.g. `fix-git`: its GitHub repo was deleted; `leelachess0`: file 404).

Source: `make_figures.py` → `data.json` (corpus block), from `terminalbench/tasks/{manifest.json, images.csv, images_docker.csv, oracle.csv, route_by_image.json}`; checked 4 Oct 2026.

KernelBench: write a faster CUDA kernel, graded on a real GPU

- **The task:** given a reference PyTorch `Model`, write `ModelNew` in `solution.py`: same outputs (atol/rtol 10^{-2}), on CUDA, ideally faster (custom kernels, fusion, better ops).
- **Rows:** 40 level-1 tasks (single ops) for training, 10 level-2 tasks (fused ops) held out. They are **mixed into the same batch** as Terminal-Bench rows.
- **The sandbox:** one **AgentCore g6 Instance** (g6.xlarge, NVIDIA L4) per episode, from the capacity provider `acs_tbh_gpu_cp`. No Terminal-Bench task needs a GPU.
- **The checker** (hidden, random name, nonce'd `PASSED` line) builds `ModelNew` next to `Model`, checks shape, values and device, and prints `speedup=`.

Table 4 **Reward (`TB_KB_SPEEDUP_MIN` = 1.10)**

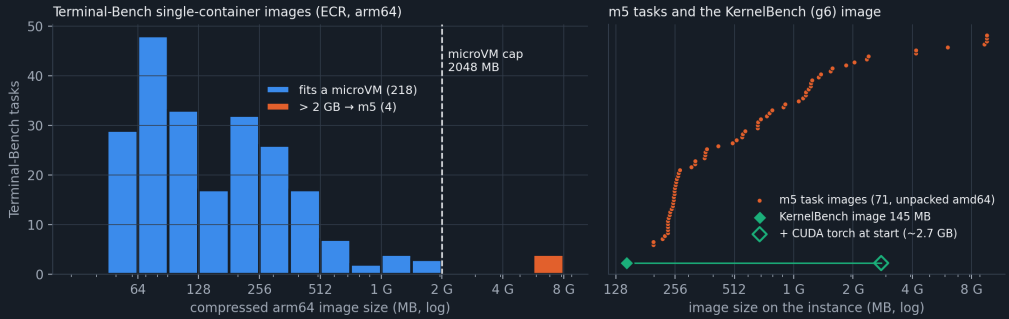
Outcome	Reward
correct and $\geq 1.10\times$ faster	**1.0**
correct, not faster	0.5
wrong, crashed or not on CUDA	0

- The first reward was correctness only: copying `Model` onto CUDA scored 1, so groups were all 1s (smoke 4: 48 of 75).
- Copies of the reference measure 1.00–1.03×, so the 1.10× bar is above timing noise.

Source: `kernelbench/`, `data/kernelbench_{train,eval}.jsonl`, `deploy/Dockerfile.kernelbench_gpu`, demo README (KernelBench on the g6 tier); checked 4 Oct 2026.

Image sizes: most tasks fit a microVM, the tail and the GPU task do not

Figure 1 **Task image sizes against the 2 GB microVM image cap**



- **arm64 microVM images:** p50 **127 MB**, p95 620 MB, largest that fits 2.0 GB.
- **4 tasks are 6.5–6.7 GB** (`mteb-*`, `png-generation`): no microVM can take them.
- **m5 images** (unpacked, amd64): p50 525 MB, up to 9.8 GB; the m5 has a 40 GB disk.
- **KernelBench:** the 2 GB cap also applies to capacity-provider Runtimes, so the image is 145 MB and installs CUDA torch (∼2.7 GB) at container start (∼90 s).

Source: `images.csv` (ECR compressed size), `images_docker.csv` (`docker image inspect`); `make_figures.py`; checked 4 Oct 2026.

The demo: use case covered, why this demo, design considerations

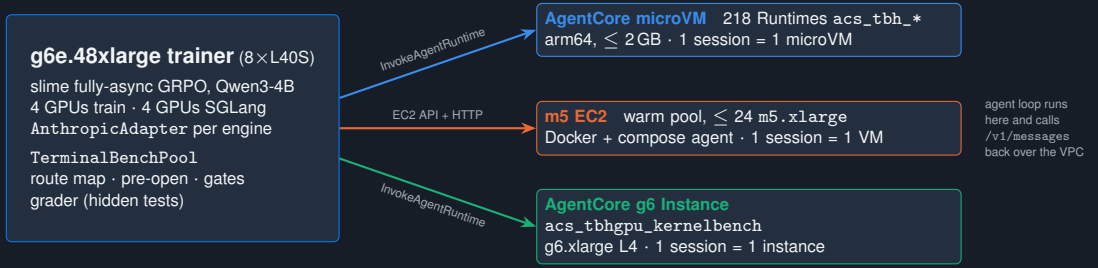
Table 5 **The demo in three answers**

Use case covered	Agentic RL on a benchmark whose environments **do not all fit one sandbox product**: most tasks are one small container, but some need several networked containers, an x86 CPU, a multi-GB image, root powers a capability-less microVM does not grant, or a **GPU** (KernelBench).
Why this demo	It trains on **the whole usable corpus** (193 Terminal-Bench tasks + 50 KernelBench) instead of the 165 the microVM pool alone can validate. It shows the sandbox tier can be **heterogeneous without touching the RL side**: agent loop, grader, reward, pre-open and metrics are one code path for all three substrates. `eval/terminalbench` stays the same held-out set as `demo_cpu_terminalbench`.
Design considerations	**One protocol, three transports:** every image is the task's own Dockerfile plus the same wrapper (standalone `/opt/acs/py` + stdlib `acs_http_server.py`); AgentCore invokes it on a microVM or g6, the trainer POSTs to it on an m5 (`DockerComposeSandbox` only swaps the transport) · **routing is data**, not a hand-kept list (slide 5) · **one session per sandbox**: a fresh microVM, a warm m5 reused only after a **verified** reset, a fresh g6 Instance · **per-episode compose rewrite**: fixed container names, host ports and `./` mounts made unique, privileged/host-network/`docker.sock` refused · **hidden tests stay hidden**: task dirs reach the m5 without `tests/` or the solution · **rejected:** running compose on AgentCore (no capabilities, no Docker daemon), the g6e as Docker host (it trains), g5 for KernelBench (accepted, but g6 is cheaper: $0.80 vs $1.01/h).

Source: `code/demos/demo_hybrid_terminalbench/README.md`; `docs/terminalbench-hybrid-demo-plan.md`; checked 4 Oct 2026.

One trainer, one router, three sandbox substrates

Figure 2 **Components: the trainer routes each row; the agent loop runs inside the sandbox and dials back**



microVM: 163 tasks. Managed by AgentCore; an idle session is wiped after 15 min, so a heartbeat keeps pre-opened ones alive.

m5: 30 tasks. **Our code** launches and terminates them (boto3, the g6e's role), from a baked AMI; images from ECR.

g6: KernelBench. In an AgentCore-managed account: **not in our EC2 console**; one CloudWatch log stream per session.

Source: `agentcore_slime_demo/{terminalbench_pool, docker_compose_sandbox, ec2_episode_sandbox}.py`, `image_routing.py`, `run.sh`; checked 4 Oct 2026.

One episode is the same five steps on every substrate

Table 6 What `TerminalBenchPool` does per episode, by substrate

Step	microVM	m5 EC2 (warm)	g6 Instance
1. Route	`arn:.../acs_tbh_<task>`	`docker://<ip>:18300/<task>`	`gpu:arn:.../acs_tbhgpu_kernelbench`
2. Open (ahead)	new `runtimeSessionId`; first invoke boots the microVM	idle VM (prefer one holding the images) or `run_instances`; ECR pull; `compose up`	new session id; first invoke launches a g6, then torch install
3. Agent loop	mini_claude (`bash` / `write_file` / `read_file`), up to 8 turns × 1024 tokens, **inside the sandbox**, calling the trainer's adapter		
4. Grade	inject `tests/`, run pytest	same, in the client	hidden checker: correctness + `speedup=`
5. Close	`StopRuntimeSession`	`compose down -v` + `reset_check`; clean → pool, dirty → terminate	`StopRuntimeSession` (instance released)
Open cost (smoke 6, p50)	**2 s**	**68 s** (cold launch 34–51 s + pull)	**135 s** (boot + ∼90 s torch)
Episode p50	24 s (agent 12 s, grade 9 s)	69 s (agent 14 s, grade 7 s)	–

In plain English: the RL code never asks which substrate it is on; only the open cost differs (2 s to over 2 min).

Source: `logs/smoke6-hybrid_*.log` (`[demo] agent_ms, grade_ms, episode_ms, open_ms`); m5 per-episode probe; checked 4 Oct 2026.

Sessions and instances: one session per instance, three lifecycles

Table 7 Who owns each instance, and how it ends

	microVM	m5	g6
Managed by	AgentCore	**our trainer**	AgentCore
Reused?	no, fresh	yes, after a verified reset	no, fresh
Isolation	own kernel	own VM per session	own instance
Visible in	CloudWatch	our EC2 console	CloudWatch only
Ends by	stop / 15-min idle	5 ways (right)	stop / 600 s idle

Table 8 Warm m5 pool in smoke 6

instances launched / episodes on a reused one	24 / 10
pool peak (cap `TB_M5_MAX_INSTANCES`)	24 (24)
dirty resets / retired	**0 / 0**

- **Verified reset** after every m5 episode: no containers, volumes or extra networks, empty episode dir, $\geq$ 8 GB disk. Dirty or unhealthy $\rightarrow$ terminated, never reused. No periodic recycle.
- **m5 termination, 5 ways:** trainer `atexit`; `5_run.sh gc-episodes`; next pool start; `6_cleanup.sh`; a **dead-man switch** (no trainer call for 30 min $\rightarrow$ the VM shuts itself down = terminate).
- **m5 hardening:** no IAM role; IMDSv2 hop 1 and blocked from containers; containers cannot reach the host; agent API needs a secret and the trainer's SG.
- **g6:** a launch per session, so a pre-open must start $\sim$2.5 min early (slide 12).

Source: `agentcore_slime_demo/ec2_episode_sandbox.py` (`WarmPool`), `m5_agent/bootstrap.sh`; `logs/smoke6-hybrid_*.log` (`warm_reused`, `warm_instances`); checked 4 Oct 2026.

Pre-open hides cold starts from 2 s to 2.5 min behind the running batch

Figure 3 Median wait for the sandbox at episode start: measured with pre-open vs the open time it hides

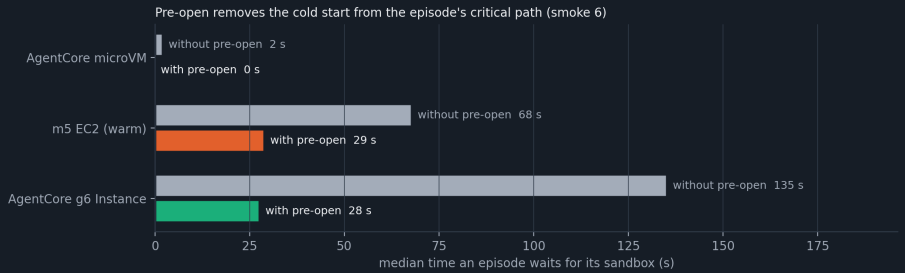


Table 9 Smoke 6, by substrate

	n	no wait	wait p90	saved
microVM (2 batches ahead)	177	121	5.6 s	4 min
m5 (5 batches ahead)	34	16	61 s	23 min
g6 (5 batches ahead)	52	22	100 s	**80 min**

Table 10 g6 session cap vs start wait

Run	cap	wait p50	no wait
smoke 4	8	240 s	0 / 76
smoke 5	16	82 s	5 / 36
smoke 6	32	**28 s**	22 / 52

A cap of 8 equals one group, so nothing could open ahead.

Source: `logs/smoke{4mix,5,6}-hybrid_*.log` (`[demo] open_wait_ms, open_ms`); "saved" = $\sum$ max(0, open − wait); `make_figures.py`; checked 4 Oct 2026.

Five smoke runs: the pipeline works on all three substrates

Table 11 **Smoke runs on 4 Oct 2026 (train + eval episodes; solved / episodes; infra errors)**

Run	microVM	m5	g6 KernelBench	Eval and change
smoke 2	17 / 401; 1 err	0 / 88; 0	–	TB 2/62; final ckpt save hung → fixed
smoke 3	16 / 382; **0**	0 / 88; **0**	–	**passed**; TB 4/62, m5 0/8
smoke 4 (mix)	13 / 322; 1	0 / 58; 1	48 / 75; 0	**passed**; KB 16/20 → reward too easy
smoke 5	6 / 157; 0	0 / 34; 0	26×0.5; 0	**passed**; speedup-gated reward, g6 cap 16
smoke 6	4 / 177; 1	0 / 34; 0	8×1, 34×0.5; 0	**passed**; KB mean 0.45, up to 1.38×; cap 32

- **Infra is solved:** 4 infra errors in ∼1,900 episodes over 5 runs, all three substrates.
- **KernelBench gives a gradient:** with the gated reward, groups mix 0, 0.5 and 1.
- **The m5 tasks are hard:** 0 solved in 302 episodes. They are the compose, x86 and big-image tail.
- **Terminal-Bench eval** on the microVM set: 1–4 of 62, as in `demo_cpu_terminalbench`.

Source: `logs/smoke{2,3,4mix,5,6}-hybrid_*.{log,eval.csv}`; `make_figures.py` → `data.json` (runs); checked 4 Oct 2026.

The 48-step training run is in progress

Table 12 `train48`**, started 4 Oct 2026 15:24 UTC (status at 16:24)**

steps done	19 of 48
mean step time	130 s
episodes / solved	935 / 108
infra errors	1
expected end	≈ 17:50 UTC + eval

Table 13 **Train reward per step (**`rollout/raw_reward`**)**

steps 0–9: 0, 0, .25, .25, 0, .13, .11, 0, .02, 0

steps 10–18: .08, 0, 0, .22, 0, .13, 0, .13, 0

- Noisy and flat so far, as expected at this point.
- Final rewards and eval go here when the run ends.

Source: `logs/train48-hybrid_20261004_152433.log`; snapshot 4 Oct 2026 16:24 UTC.

Conclusion: every task gets a sandbox it runs on; the reward is what is left

Table 14 What the demo shows

	Conclusion	Evidence	Slides
1	**A hybrid tier covers the corpus**	193 usable Terminal-Bench tasks vs 165 on microVMs alone, plus KernelBench on g6	slides 5–7
2	**The RL side does not change**	one agent loop, grader, pre-open and metrics path for microVM, m5 and g6	slides 9–10
3	**Infra is reliable**	4 infra errors in ∼1,900 smoke episodes; 0 dirty warm resets	slides 11–13
4	**Pre-open depth must match boot time**	g6 start wait p50 240 s → 28 s with cap 32 and 5 batches ahead	slide 12
5	**The reward limits learning**	TB eval 1–4 of 62; 0 of 302 m5 episodes solved; KB needed a speedup gate	slide 13

Table 15 Next steps

1	Finish the 48-step run	running (slide 14)
2	Partial credit from `tests_passed` / `tests_total`	more GRPO groups with a gradient, esp. on m5 tasks
3	Host the CUDA torch wheel in S3	cut the g6 open from ∼2.5 min
4	Stop the m5 builder when not building	it is not needed for training

Source: This deck; checked 4 Oct 2026.

Thank you

code/demos/demo_hybrid_terminalbench